

# Scan QR Code For Attendance

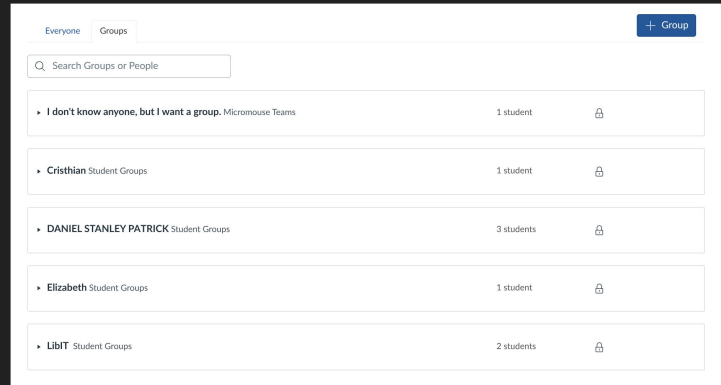


# Micromouse

## Lecture 4: Floodfill

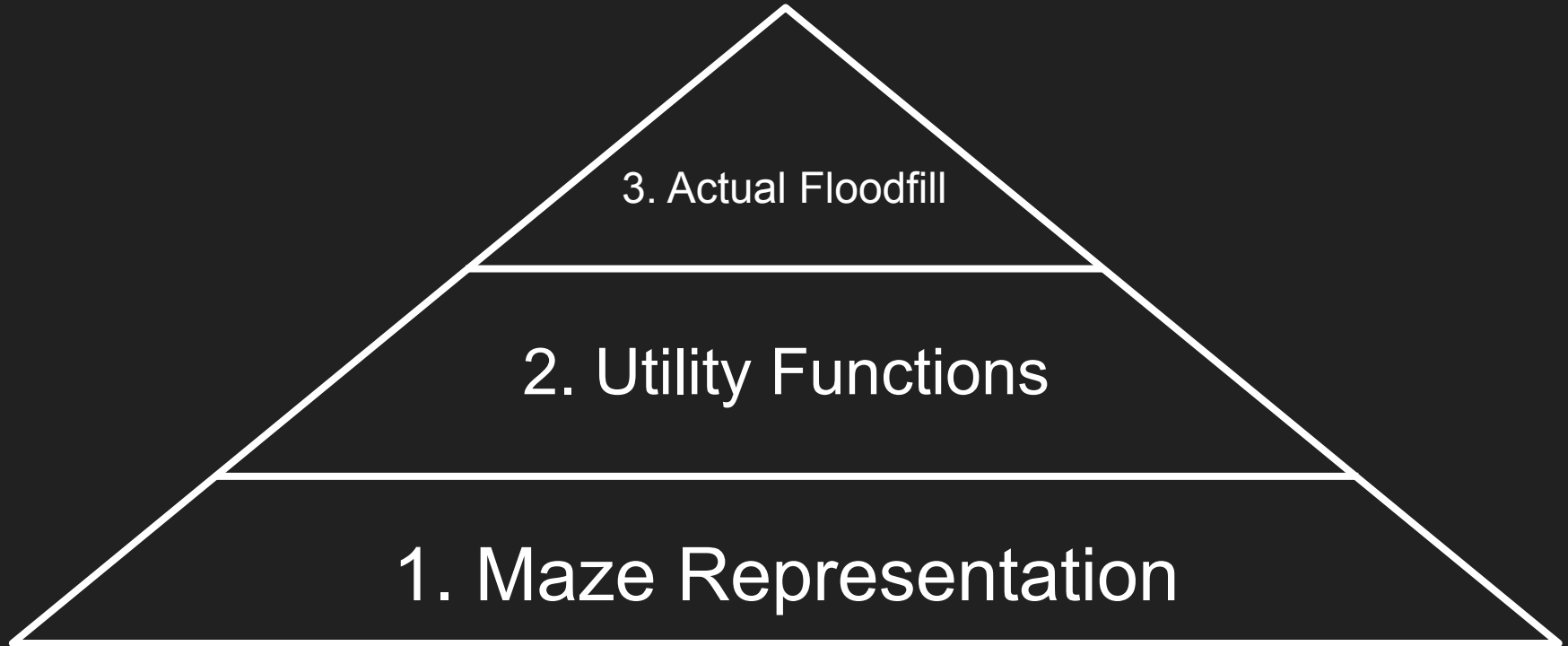
# Announcements

- **Assignments due TOMORROW**
  - Team formation
  - Final PCB design (1 submission per team)
- We will be doing final PCB reviews & meet with people who need to revise things



How to: Floodfill

# Steps Of Implementation



# Maze Representation

```
Coord mousePos
```

```
Direction mouseDir
```

```
struct Maze:
```

```
    int distances[16][16]
```

```
    int cellWalls[16][16]
```

```
    Coord* goalPos
```

```
struct Coord:
```

```
    int x,
```

```
    int y
```

```
enum Direction:
```

```
    NORTH = 0,
```

```
    WEST = 1,
```

```
    SOUTH = 2,
```

```
    EAST = 3
```

```
struct Cell:
```

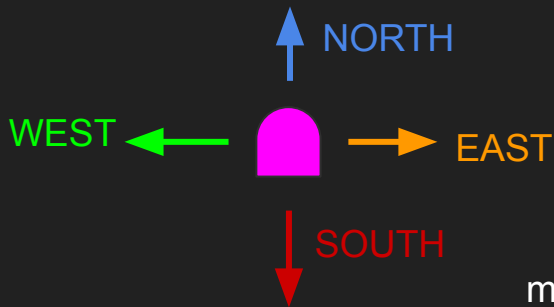
```
    Coord pos,
```

```
    Direction dir,
```

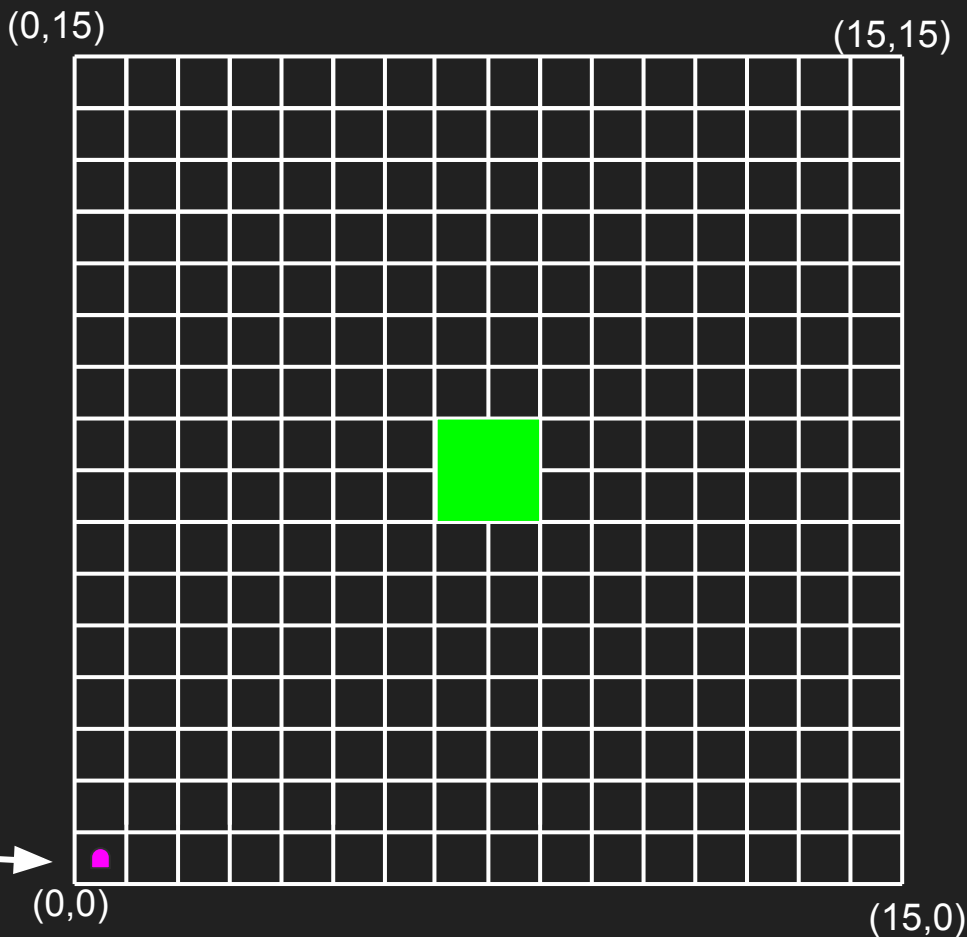
```
    bool blocked
```

# Maze Representation

```
Coord mousePos  
Direction mouseDir  
struct Maze:  
    int distances[16][16]  
    int cellWalls[16][16]  
    Coord* goalPos
```



mousePos = (0,0) →



# Maze Representation

```
Coord mousePos
Direction mouseDir
struct Maze:
    int distances[16][16]
    int cellWalls[16][16]
    Coord* goalPos
```

Represent the walls in a cell as a number

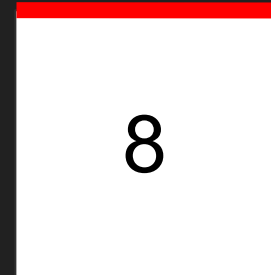
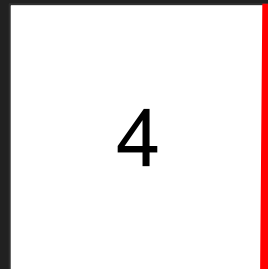
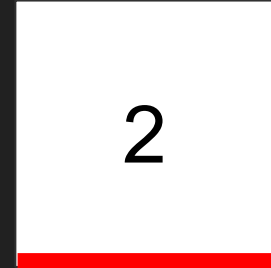
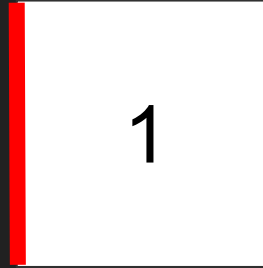
Ex.

West = 1

South = 2

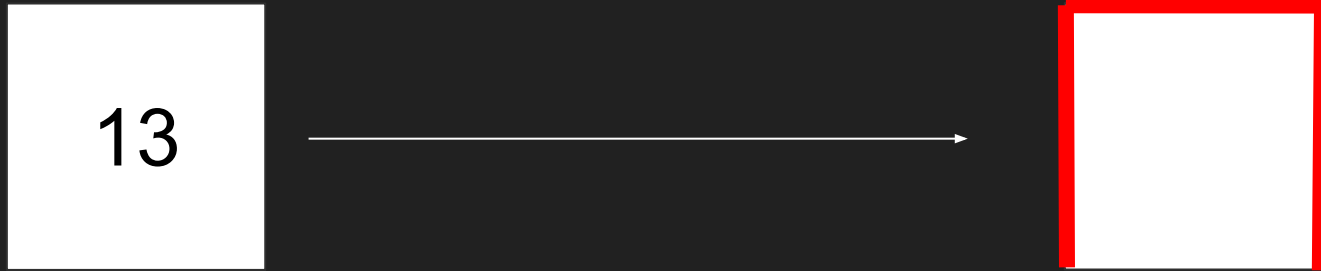
East = 4

North = 8

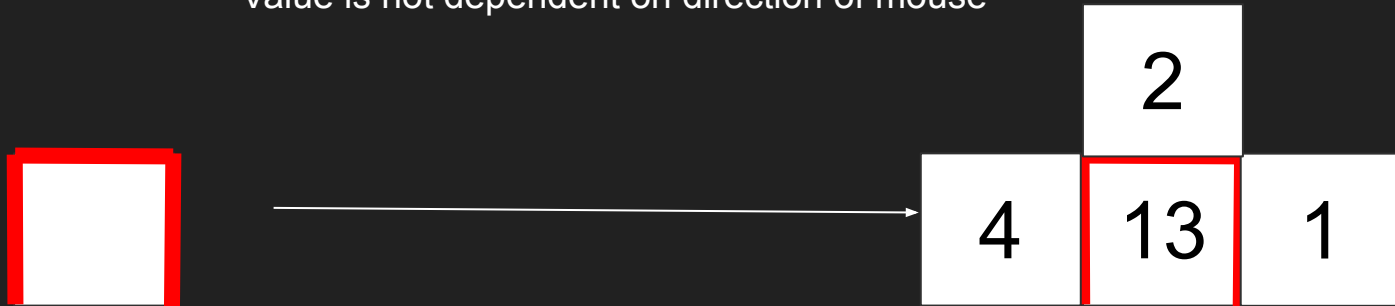




# Maze Representation



Value is not dependent on direction of mouse



# Maze Representation - Binary

North Wall = 8 -> 1000

East Wall = 4 -> 0100

South Wall = 2 -> 0010

West Wall = 1 -> 0001

13 -> 1101 -> N, E, W walls

7 -> 0111 -> E,S,W walls

12 -> 1100 -> N,E walls

# Maze Representation

Where are the walls for this set of cells?

North Wall = 8 -> 1000

East Wall = 4 -> 0100

South Wall = 2 -> 0010

West Wall = 1 -> 0001

|      |      |      |
|------|------|------|
| 1011 | 1000 | 1100 |
| 1111 | 0101 | 0101 |
| 1010 | 0110 | 0111 |

# Maze Representation

|      |      |      |
|------|------|------|
| 1011 | 1000 | 1100 |
| 1111 | 0101 | 0101 |
| 1010 | 0110 | 0111 |

# Maze Representation

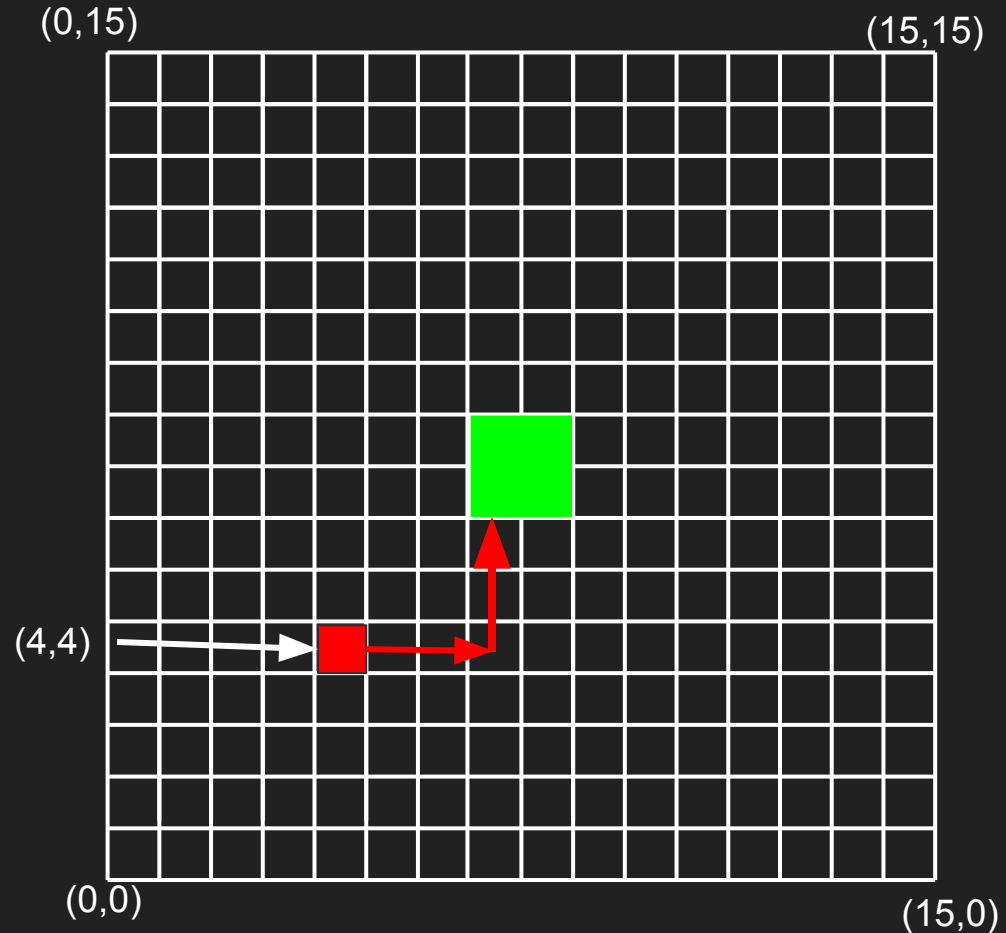
```
int distances[16][16]
```

```
distances[4][4] = 6
```

Calculating the distance:

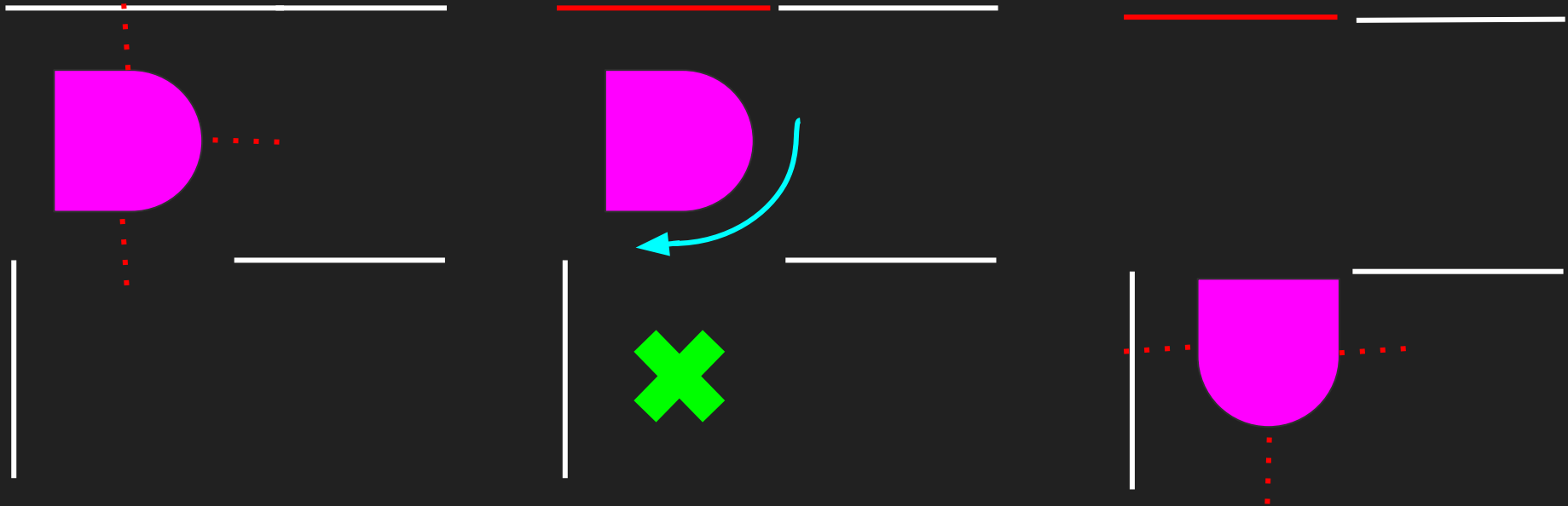
Closest Goal: (7, 7)

$$|7 - 4| + |7 - 4| = 6$$



# Main Idea Of Run Loop

1. Mouse scans the walls in its current cell
2. Finds the best cell it can move to
3. Moves to that cell



# Utility Functions

```
Cells* getNeighborCells() // differentiate between accessible cells  
                           // and cells that are blocked by walls
```

```
Cell getBestCell() // returns best accessible cell for the mouse to move  
to
```

```
// functions to return direction after step rotation
```

```
Direction clockwiseStep()
```

```
Direction counterClockwiseStep()
```

```
void setGoalCell() // sets a certain cell pos as the target cell
```

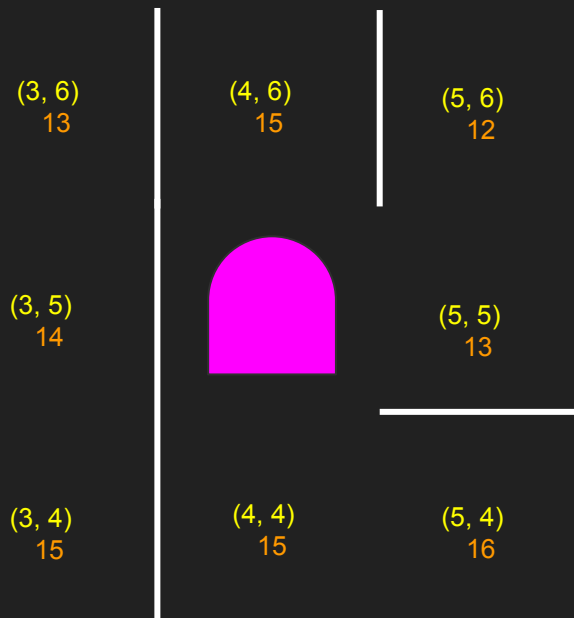
```
void rotate()
```

```
void move()
```

```
void updateSimulator() // Only updates the simulator itself
```

# Utility Functions

Example:



What to we do:

1. `Cells*` `getNeighborCells()`  
→ { (4, 6), (5, 5), (4, 4), (3, 5) }
2. `Cell` `getBestCell()`  
→ (5, 5) // since lowest distance
3. Move Mouse
  - Rotate Clockwise
  - Move Forward One Step
4. Update Simulation Display



# Utility Functions

## Priority List:

1. `updateSimulator(), clockwiseStep(), counterClockwiseStep()`
2. `updateWalls()`
3. `getNeighborCells()`
4. `getBestCell()`
5. `rotate(), move()` // see the sample left wall follower code
6. `setGoalCell()`

# FloodFill Function Pseudo Code

Floodfill:

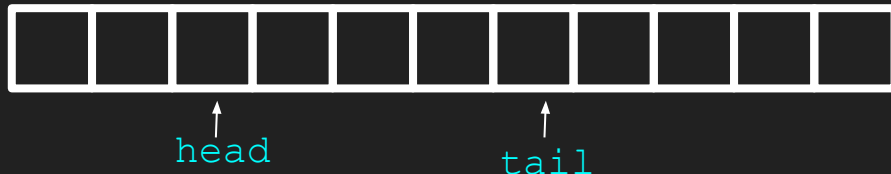
```
for(every cell):
    distances[y][x] = MAX_COST
for (all the goal cells)
    distances[y][x] = 0

head, tail = 0
for(all goal cells)
    queue.add(goal cell position)
    tail++

while(tail - head > 0):
    cur_pos = queue[head]
    head++
    newcost = distances[pos.y][pos.x] + 1

    neighborCells = getNeighborCells(pos)
    for(cell in neighborCells):
        if (cell is not blocked):
            if(distances[cell.y][cell.x] > newcost):
                distances[cell.y][cell.x] = newcost
                queue[tail] = cell
                tail++
```

```
Coord queue[255];
// first in first out
int head = 0;
int tail = 0;
```



# Set All Distance Values to Max

```
for(all distance cells):  
    distances[y][x] = MAX_COST
```

|     |     |     |
|-----|-----|-----|
| 255 | 255 | 255 |
| 255 | 255 | 255 |
| 255 | 255 | 255 |

# Assign Goal Cells

```
for (all the goal cells)
    distances[y][x] = 0

head, tail = 0
for(all goal cells)
    queue.add(goal cell position)
    tail++
```

|     |     |     |
|-----|-----|-----|
| 255 | 255 | 255 |
| 255 | 0   | 255 |
| 255 | 255 | 255 |

# Begin Iteration of Queue

```
while (tail - head > 0):  
    cur_pos = queue[head]  
    head++  
    newcost = distances[pos.y][pos.x] + 1  
  
    neighborCells = getNeighborCells(pos)
```

# Iterate through neighbor cells

```
for (cell in neighborCells):  
    if (cell is not blocked):  
        if (distances[cell.y][cell.x] > newcost):  
            Distances[cell.y][cell.x] = newcost  
            queue[tail] = cell  
            tail++
```

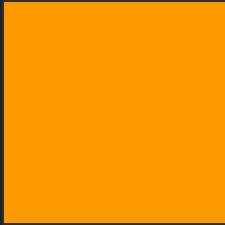
|     |     |     |
|-----|-----|-----|
| 255 | 255 | 255 |
| 255 | 0   | 255 |
| 255 | 255 | 255 |



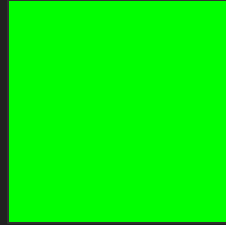
|     |   |     |
|-----|---|-----|
| 255 | 1 | 255 |
| 1   | 0 | 1   |
| 255 | 1 | 255 |

# Floodfill Demo

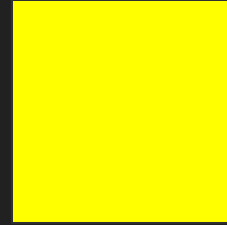
# FloodFill Examples



Current Cell



Starting Cell



Goal Cell



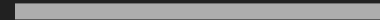
Discovered Wall



Undiscovered Wall



Maze outer wall

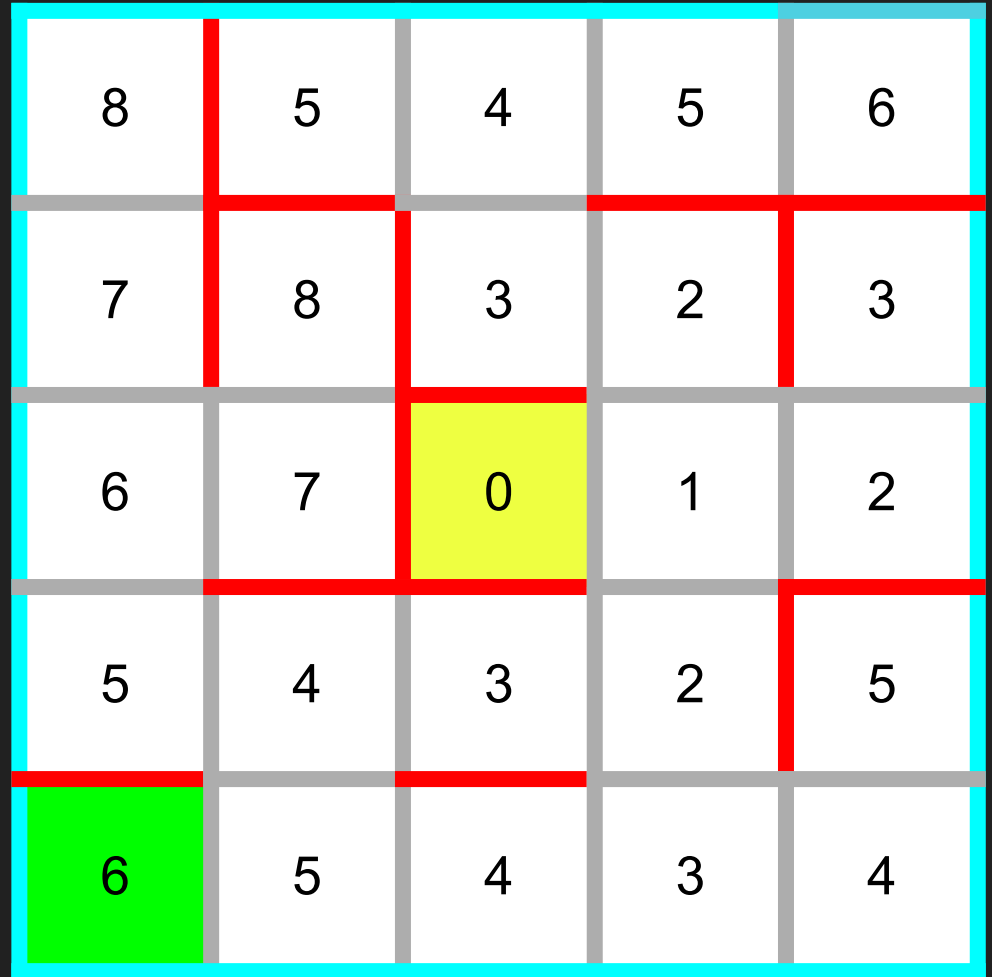


No Wall



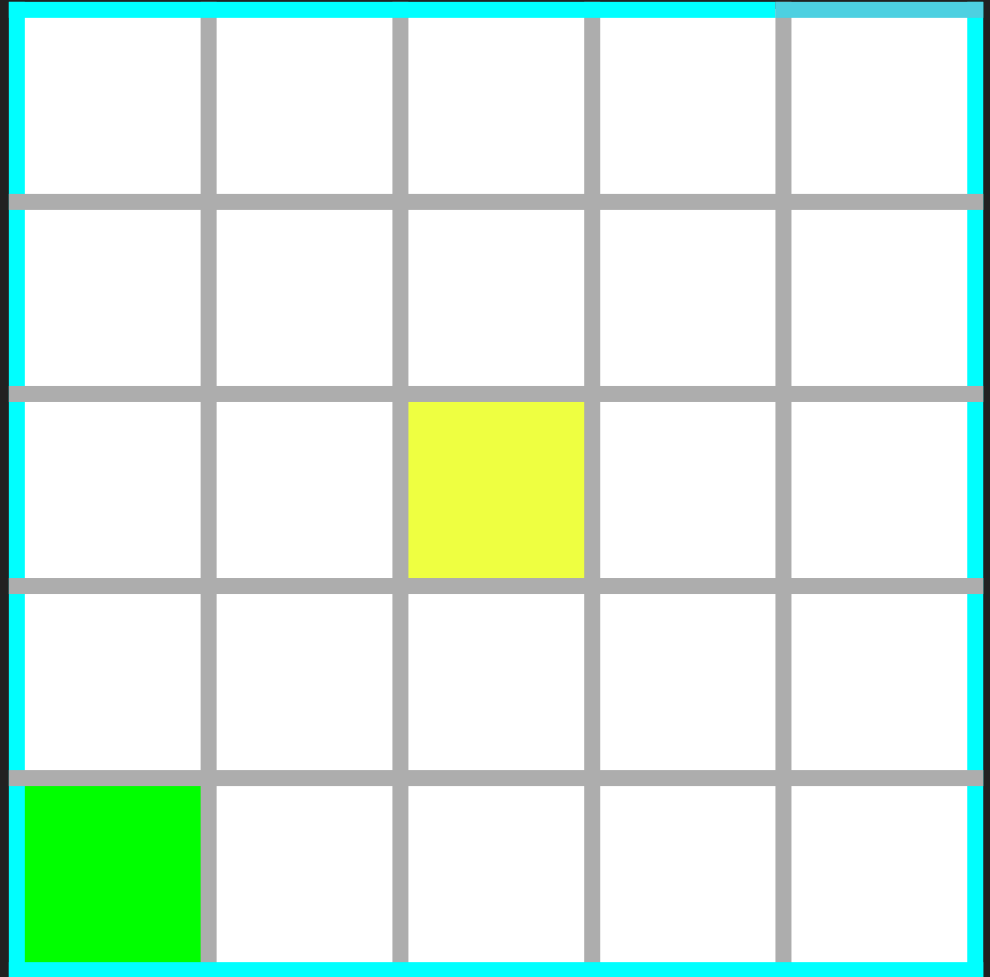
# Example 1

Completely mapped out maze with all updated distances



# Beginning

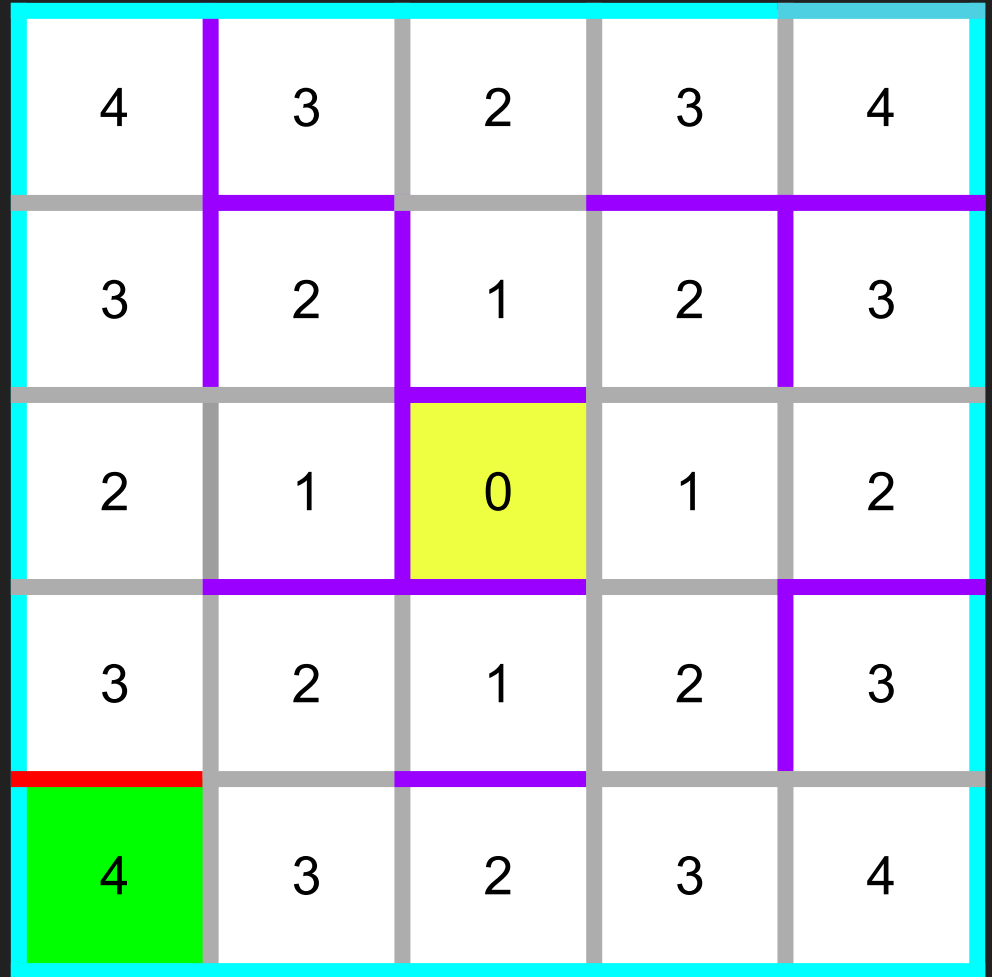
Mouse starts with blank maze



Floodfill in One Function Call

# Initialization

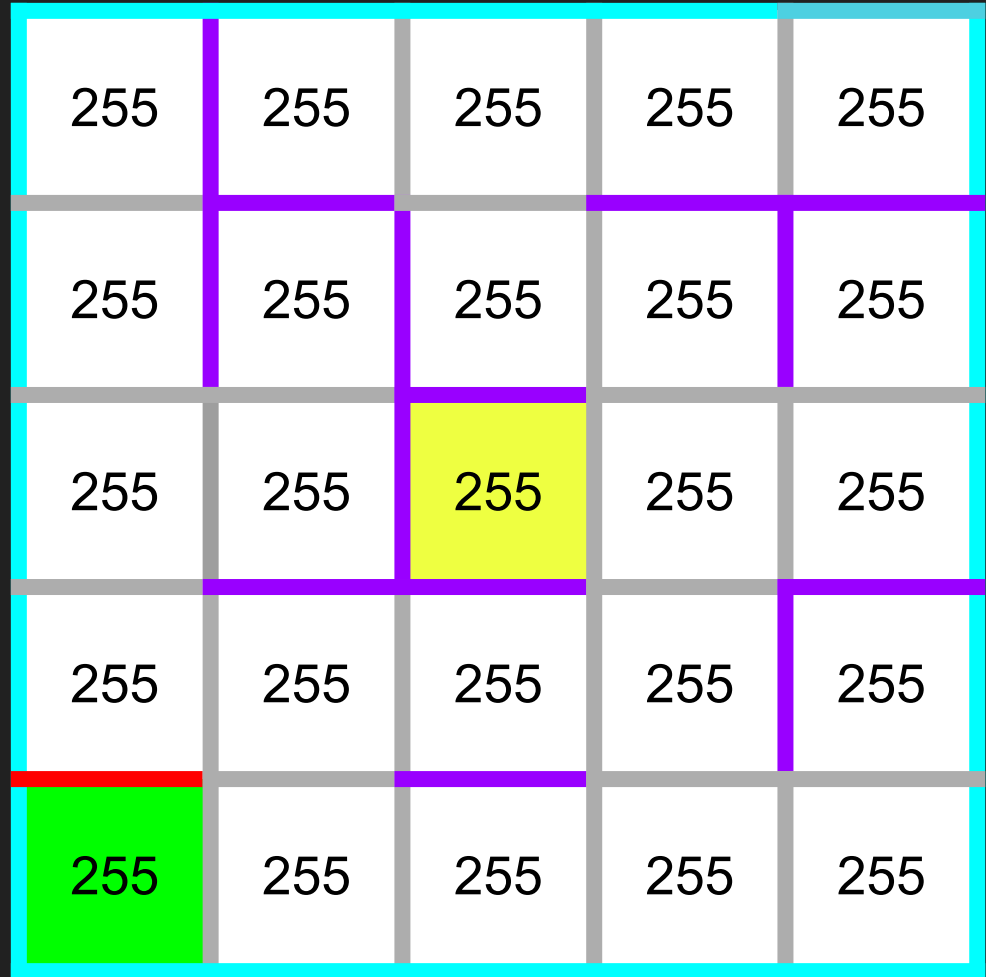
Initialize all cells with the corresponding manhattan distance.



# Resetting Distances

Calling floodfill sets all cells to Max\_cost = 255

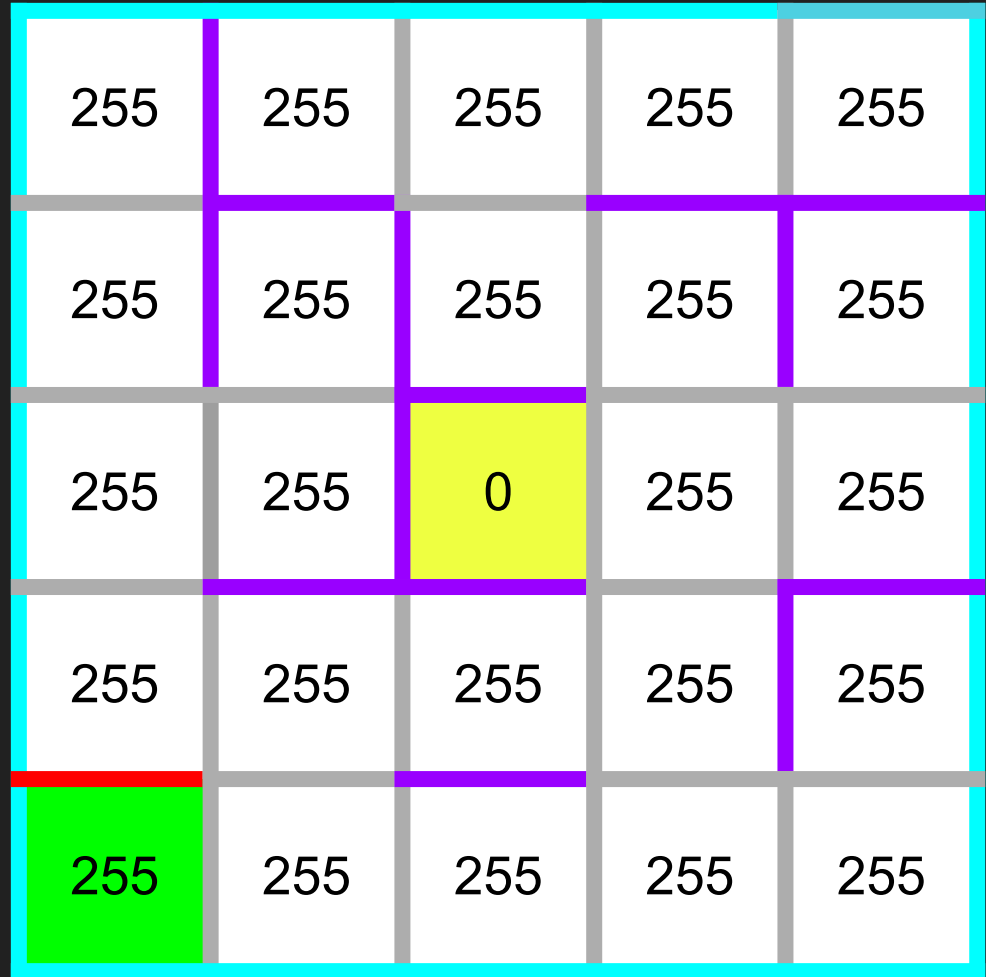
```
for (every cell):  
    distances[y][x] = MAX_COST
```



# Set Goal Cells

Set the goal cell as distance value of 0

```
for (all the goal cells)  
    distances[y][x] = 0
```



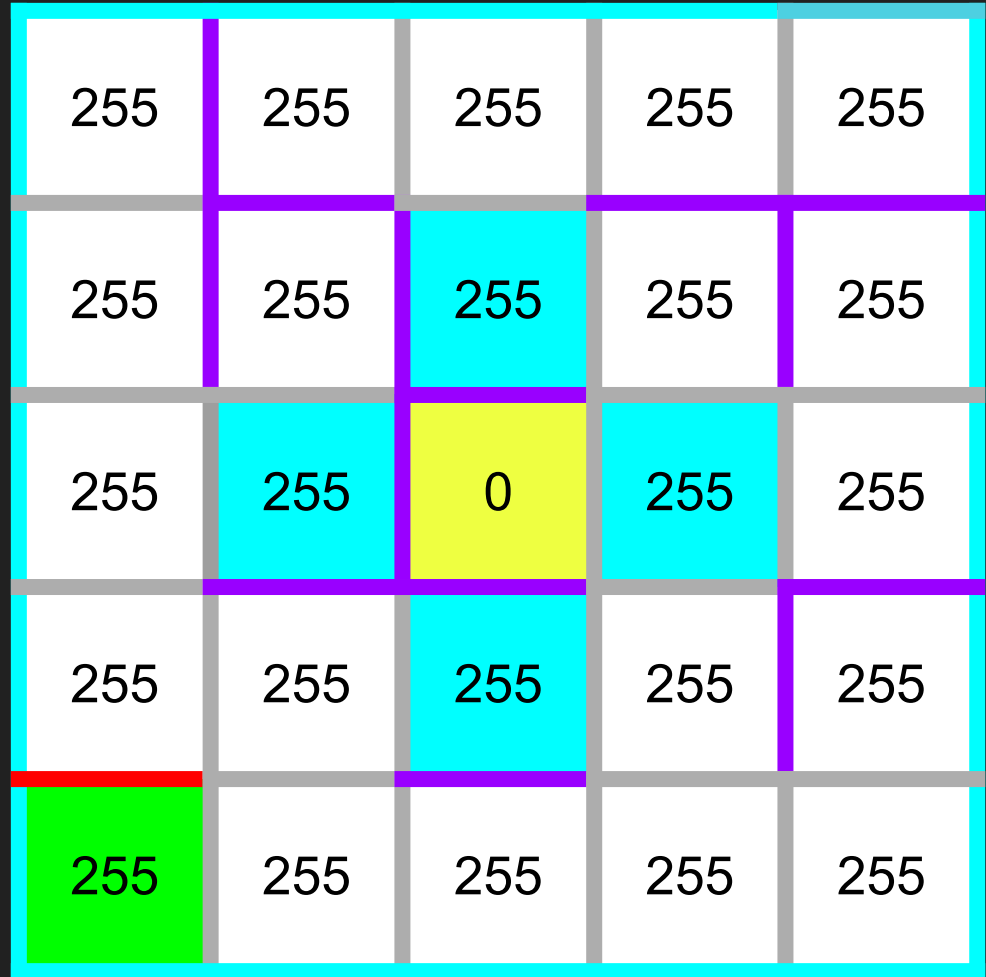
# Get Neighbors

Get neighbors of current cell

Neighbors highlighted in light blue

$\text{newCost} = 0 + 1 = 1$

```
while(queue !empty):  
    pos = queue.front()  
    newcost = distances[pos.y][pos.x] + 1  
  
    neighborCells = getNeighborCells(pos)
```



# Check Neighbors

We will check if neighbors are blocked off and update the values as necessary

```
for (cell in neighborCells):  
    if (cell is not blocked):  
        if (distances  
            [cell.y][cell.x] > newcost):  
            Distances[ce  
ll.y][cell.x] = newcost  
            queue[tail] = cell  
            tail++
```

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 255 | 255 | 255 | 255 | 255 |
| 255 | 255 | 1   | 255 | 255 |
| 255 | 1   | 0   | 1   | 255 |
| 255 | 255 | 1   | 255 | 255 |
| 255 | 255 | 255 | 255 | 255 |



# Continue checking

We will check the neighbors of the next cell in the queue

Pink cells will represent the current cell begin checked

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 255 | 255 | 255 | 255 | 255 |
| 255 | 255 | 1   | 255 | 255 |
| 255 | 1   | 0   | 1   | 255 |
| 255 | 255 | 1   | 255 | 255 |
| 255 | 255 | 255 | 255 | 255 |

# Continue checking

We will update the qualifying neighbor cells

The goal cell (0 != 2) is not enqueued again

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 255 | 255 | 255 | 255 | 255 |
| 255 | 2   | 1   | 255 | 255 |
| 2   | 1   | 0   | 1   | 255 |
| 255 | 2   | 1   | 255 | 255 |
| 255 | 255 | 255 | 255 | 255 |

Continue checking

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 255 | 255 | 255 | 255 | 255 |
| 255 | 2   | 1   | 255 | 255 |
| 2   | 1   | 0   | 1   | 255 |
| 255 | 2   | 1   | 255 | 255 |
| 255 | 255 | 255 | 255 | 255 |

Continue checking

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 255 | 255 | 2   | 255 | 255 |
| 255 | 2   | 1   | 2   | 255 |
| 2   | 1   | 0   | 1   | 255 |
| 255 | 2   | 1   | 255 | 255 |
| 255 | 255 | 255 | 255 | 255 |

Continue checking

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 255 | 255 | 2   | 255 | 255 |
| 255 | 2   | 1   | 2   | 255 |
| 2   | 1   | 0   | 1   | 2   |
| 255 | 2   | 1   | 2   | 255 |
| 255 | 255 | 255 | 255 | 255 |

Continue checking

|     |     |   |     |     |
|-----|-----|---|-----|-----|
| 255 | 255 | 2 | 255 | 255 |
| 255 | 2   | 1 | 2   | 255 |
| 2   | 1   | 0 | 1   | 2   |
| 255 | 2   | 1 | 2   | 255 |
| 255 | 255 | 2 | 255 | 255 |

Continue checking

|     |     |   |     |     |
|-----|-----|---|-----|-----|
| 255 | 255 | 2 | 255 | 255 |
| 3   | 2   | 1 | 2   | 255 |
| 2   | 1   | 0 | 1   | 2   |
| 3   | 2   | 1 | 2   | 255 |
| 255 | 255 | 2 | 255 | 255 |

Continue checking

|     |     |   |     |     |
|-----|-----|---|-----|-----|
| 255 | 3   | 2 | 3   | 255 |
| 3   | 2   | 1 | 2   | 255 |
| 2   | 1   | 0 | 1   | 2   |
| 3   | 2   | 1 | 2   | 255 |
| 255 | 255 | 2 | 255 | 255 |



Continue checking

|     |     |   |     |     |
|-----|-----|---|-----|-----|
| 255 | 3   | 2 | 3   | 255 |
| 3   | 2   | 1 | 2   | 3   |
| 2   | 1   | 0 | 1   | 2   |
| 3   | 2   | 1 | 2   | 3   |
| 255 | 255 | 2 | 255 | 255 |

Continue checking

|     |   |   |   |     |
|-----|---|---|---|-----|
| 255 | 3 | 2 | 3 | 255 |
| 3   | 2 | 1 | 2 | 255 |
| 2   | 1 | 0 | 1 | 2   |
| 3   | 2 | 1 | 2 | 255 |
| 255 | 3 | 2 | 3 | 255 |

# Continue checking

We now have a neighbor cell that is blocked, so we skip this updating

|     |   |   |   |     |
|-----|---|---|---|-----|
| 255 | 3 | 2 | 3 | 255 |
| 3   | 2 | 1 | 2 | 3   |
| 2   | 1 | 0 | 1 | 2   |
| 3   | 2 | 1 | 2 | 3   |
| 255 | 3 | 2 | 3 | 255 |

Continue checking

|     |   |   |   |     |
|-----|---|---|---|-----|
| 4   | 3 | 2 | 3 | 255 |
| 3   | 2 | 1 | 2 | 3   |
| 2   | 1 | 0 | 1 | 2   |
| 3   | 2 | 1 | 2 | 3   |
| 255 | 3 | 2 | 3 | 255 |

Continue checking

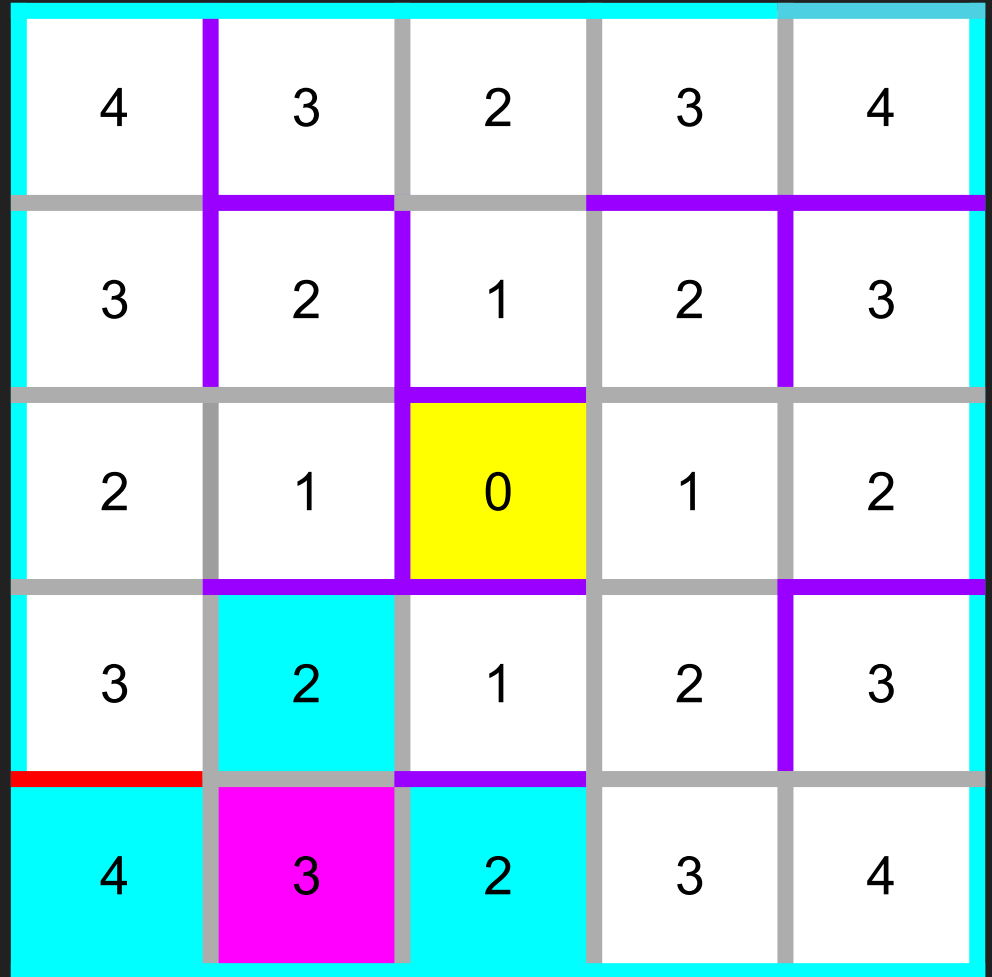
|     |   |   |   |     |
|-----|---|---|---|-----|
| 4   | 3 | 2 | 3 | 4   |
| 3   | 2 | 1 | 2 | 3   |
| 2   | 1 | 0 | 1 | 2   |
| 3   | 2 | 1 | 2 | 3   |
| 255 | 3 | 2 | 3 | 255 |

Continue checking

|     |   |   |   |   |
|-----|---|---|---|---|
| 4   | 3 | 2 | 3 | 4 |
| 3   | 2 | 1 | 2 | 3 |
| 2   | 1 | 0 | 1 | 2 |
| 3   | 2 | 1 | 2 | 3 |
| 255 | 3 | 2 | 3 | 4 |

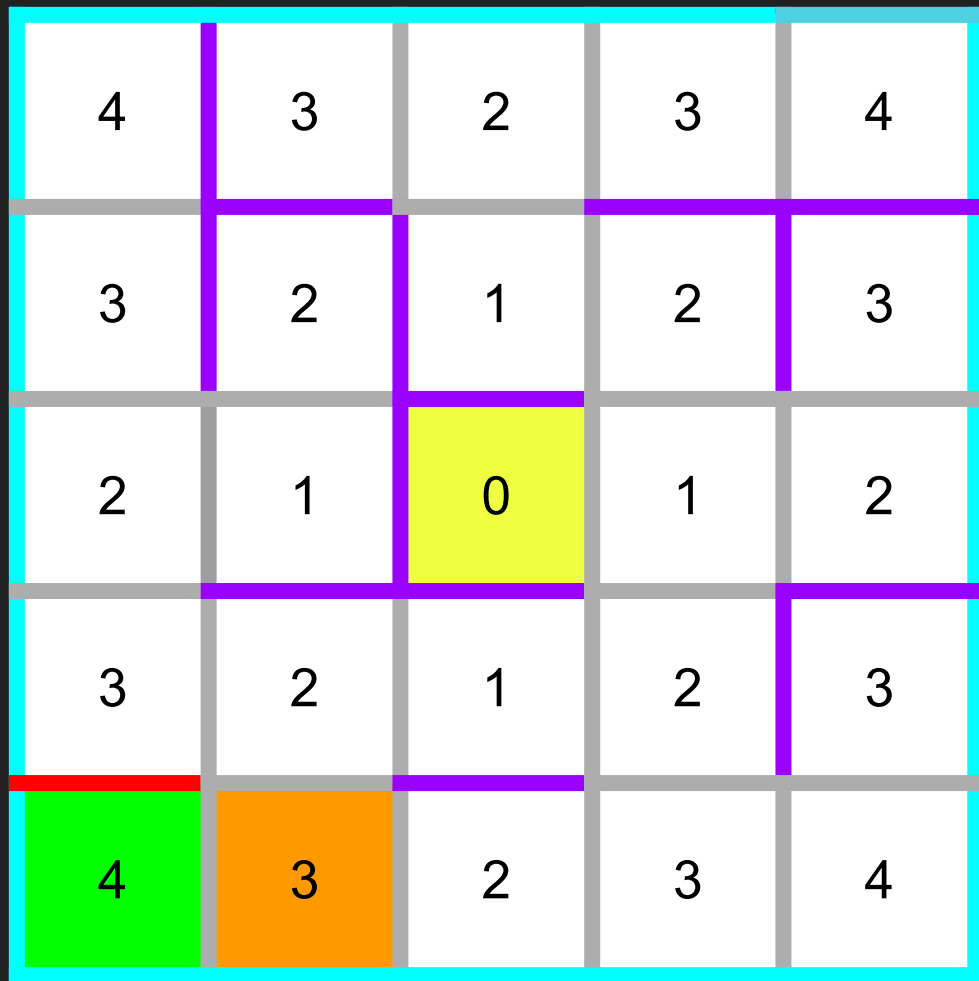
# Finish checking

Final true update of all distance values for maze



Updates and Movement through the Maze

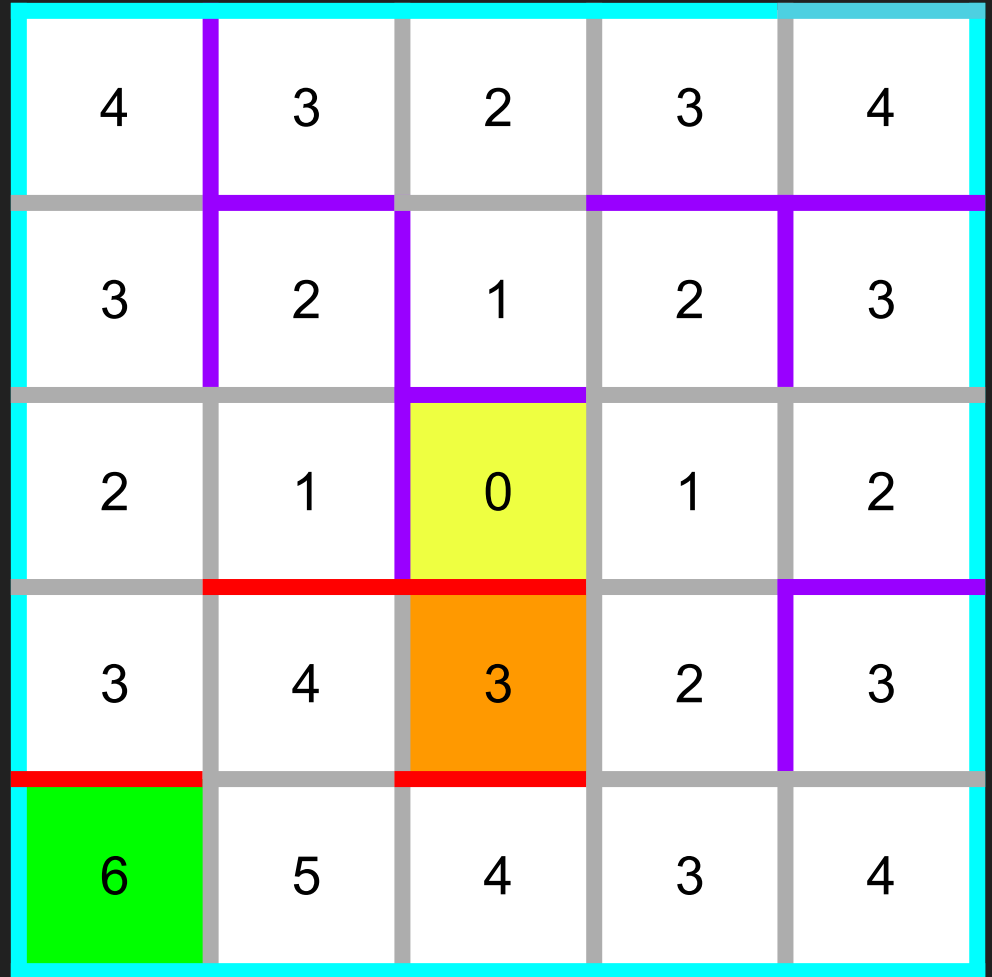




|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 3 | 2 | 1 | 2 | 3 |
| 2 | 1 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

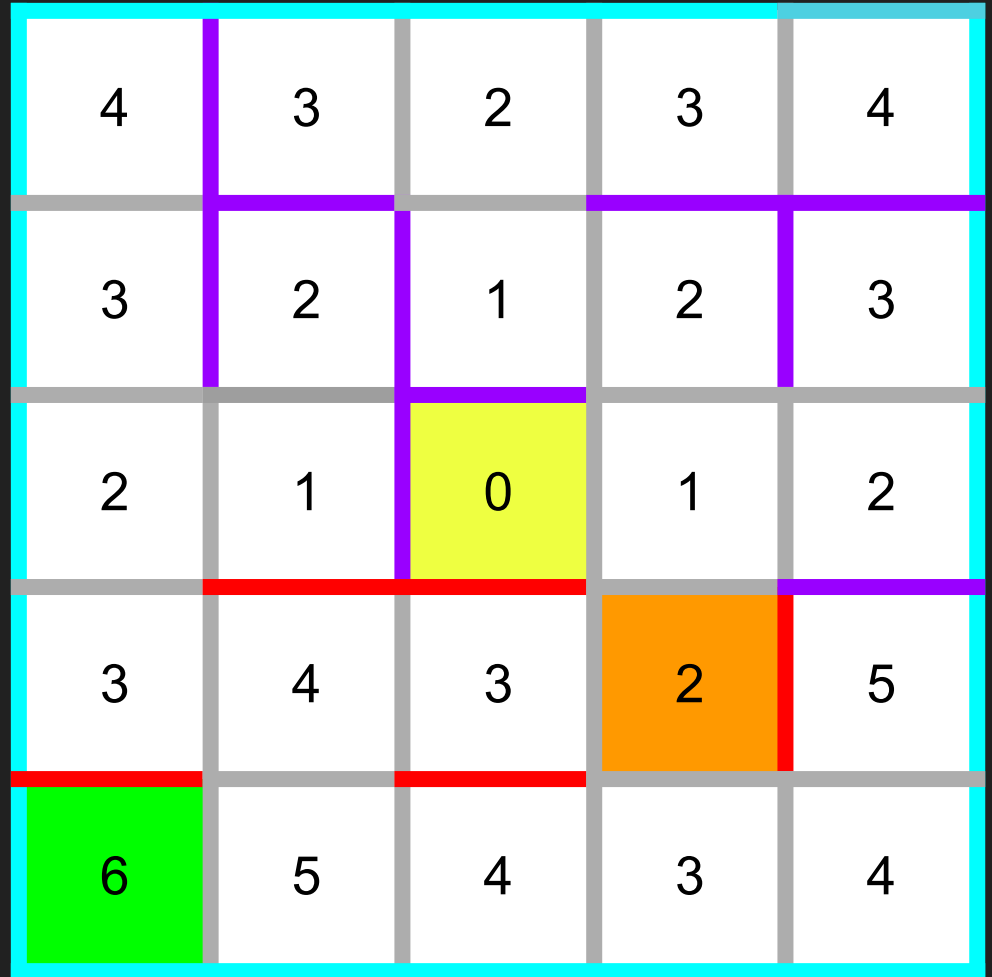
# Floodfill Update

The distance values are updated



## Continuing...

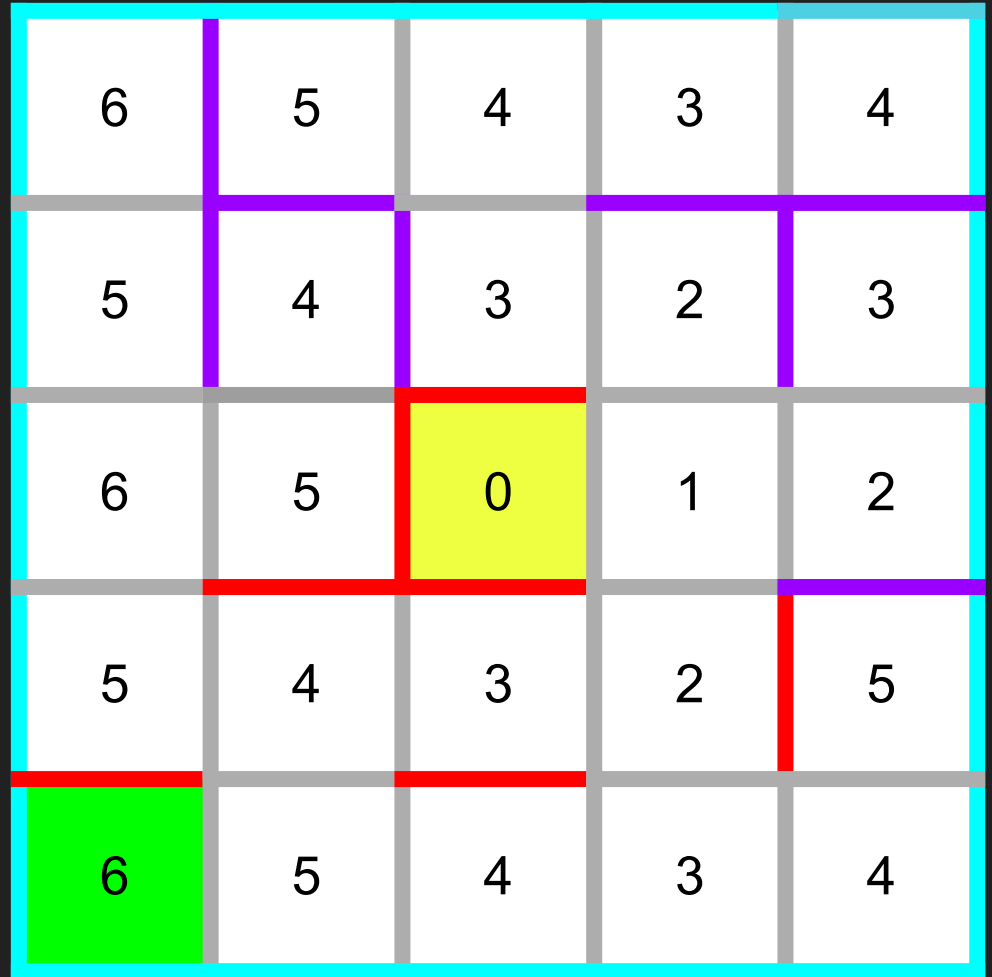
The mouse continues to go through the maze and checks for any other updates.



|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 3 | 2 | 1 | 2 | 3 |
| 2 | 1 | 0 | 1 | 2 |
| 3 | 4 | 3 | 2 | 5 |
| 6 | 5 | 4 | 3 | 4 |

# Finish

The mouse gets to the goal. Here are the final distances recorded.



Additional examples

## Example 2

A 5x5 grid with numbers. The grid is outlined in cyan. Red lines highlight a path starting from the bottom-left cell (8) and ending at the cell containing 0. The path consists of the following cells: (5,1) with value 8, (4,1) with value 8, (4,2) with value 11, (3,3) with value 0. The cell (3,3) is highlighted in yellow. The cell (5,1) is highlighted in green. The grid contains the following numbers:

|   |    |   |   |   |
|---|----|---|---|---|
| 6 | 5  | 4 | 5 | 4 |
| 9 | 10 | 3 | 2 | 3 |
| 8 | 11 | 0 | 1 | 2 |
| 7 | 6  | 3 | 2 | 3 |
| 8 | 5  | 4 | 3 | 4 |



# Beginning and Initialization

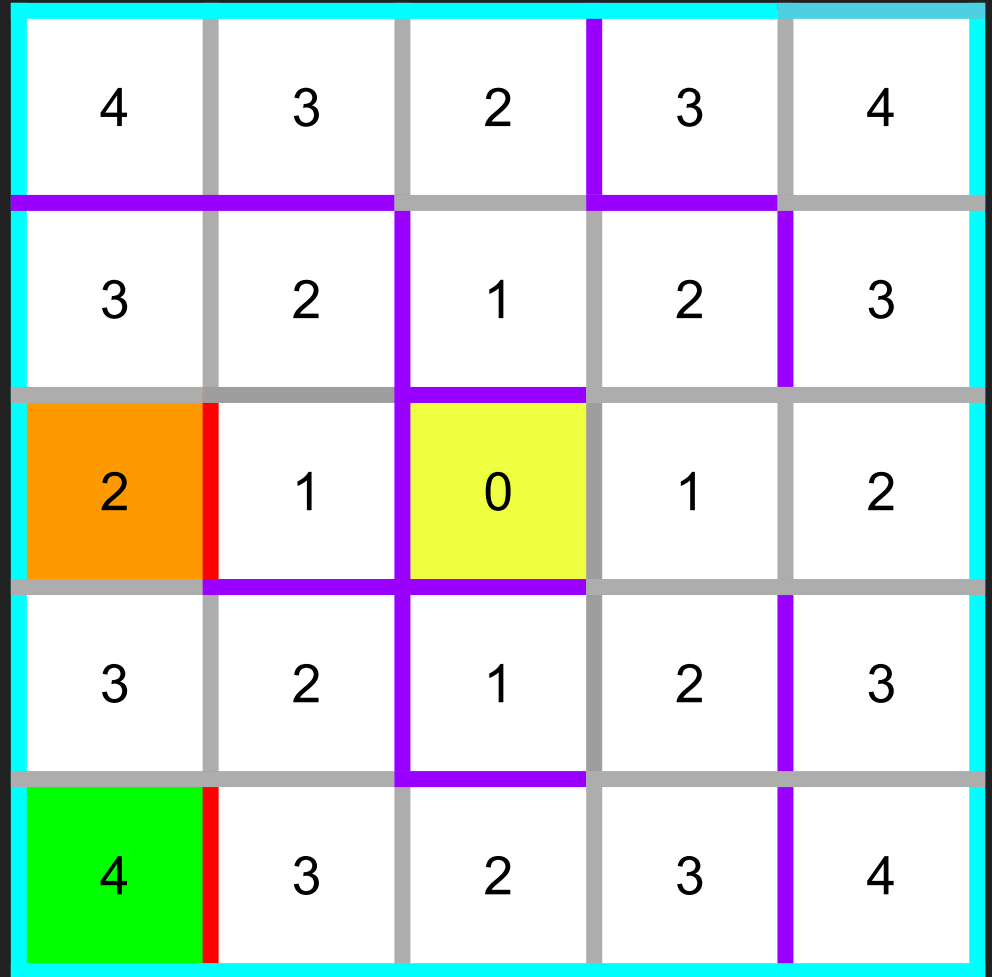
A 5x5 grid representing a game state. The grid contains numerical values from 0 to 4. The center cell (row 3, column 3) is highlighted in yellow and contains the value 0. The bottom-left cell (row 5, column 1) is highlighted in green and contains the value 4. A red vertical line is positioned between the first and second columns of the bottom row. Purple lines highlight the first and third columns of the second, third, and fourth rows. The entire grid is enclosed in a cyan border.

|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 3 | 2 | 1 | 2 | 3 |
| 2 | 1 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

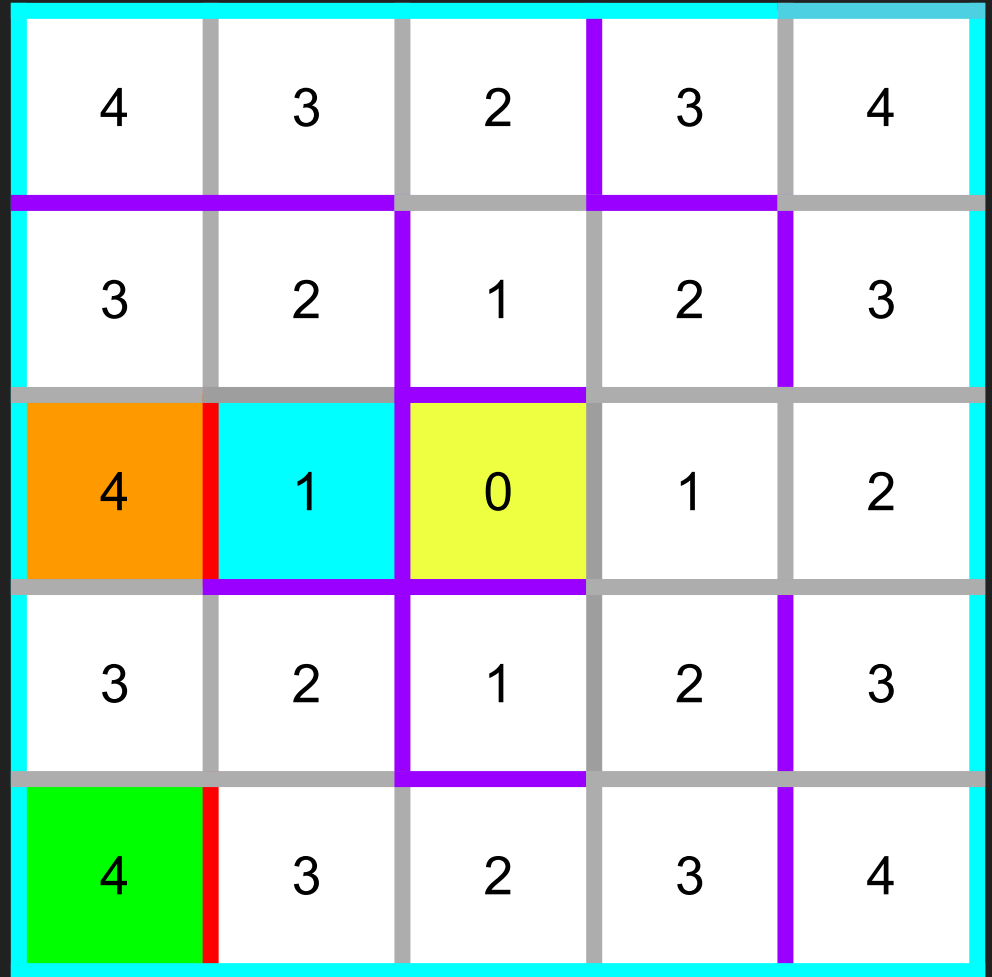
|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 3 | 2 | 1 | 2 | 3 |
| 2 | 1 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

# Updating

After the mouse moves to this cell,  
what will be the new distances?



We are blocked off so the cell our mouse is at is updated to 4

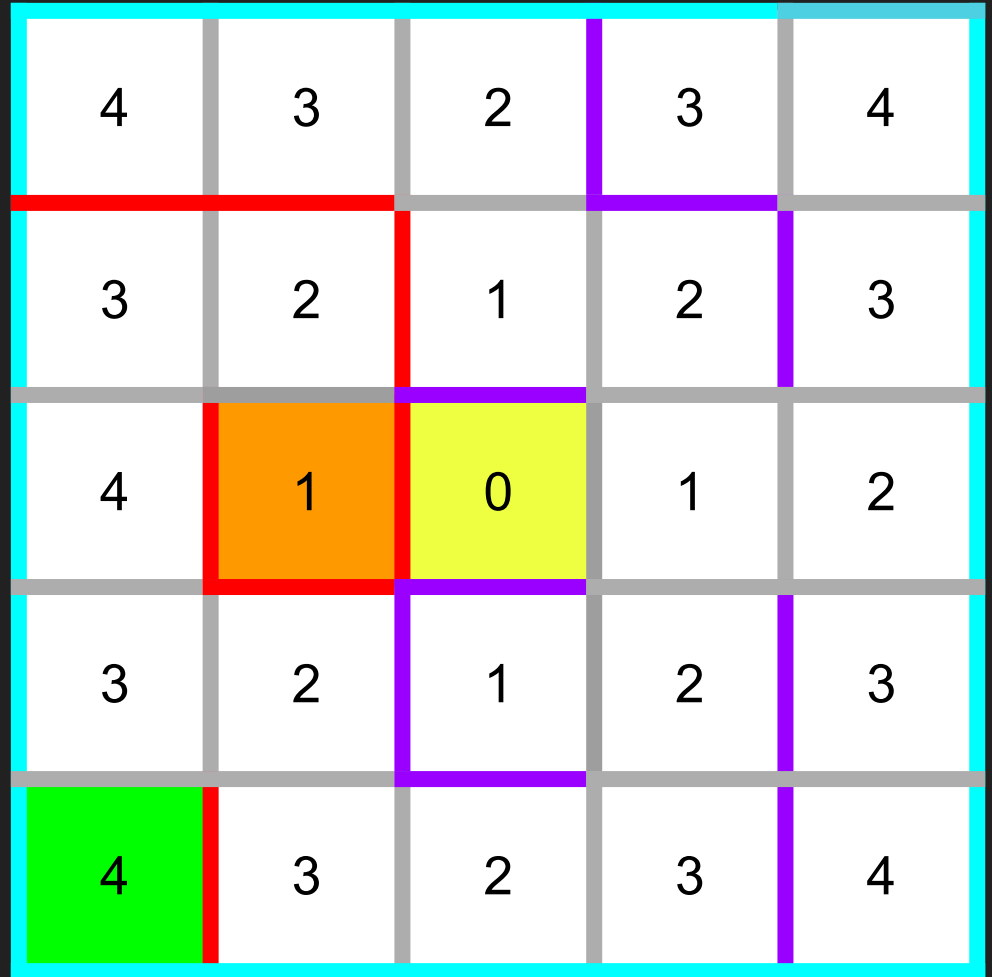


|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 1 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 1 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

# Update

Mouse discovers wall that blocks off goal cell. What will be the corresponding updated distance values?



|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 5 | 6 | 1 | 2 | 3 |
| 4 | 7 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |



|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 5 | 6 | 1 | 2 | 3 |
| 4 | 7 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

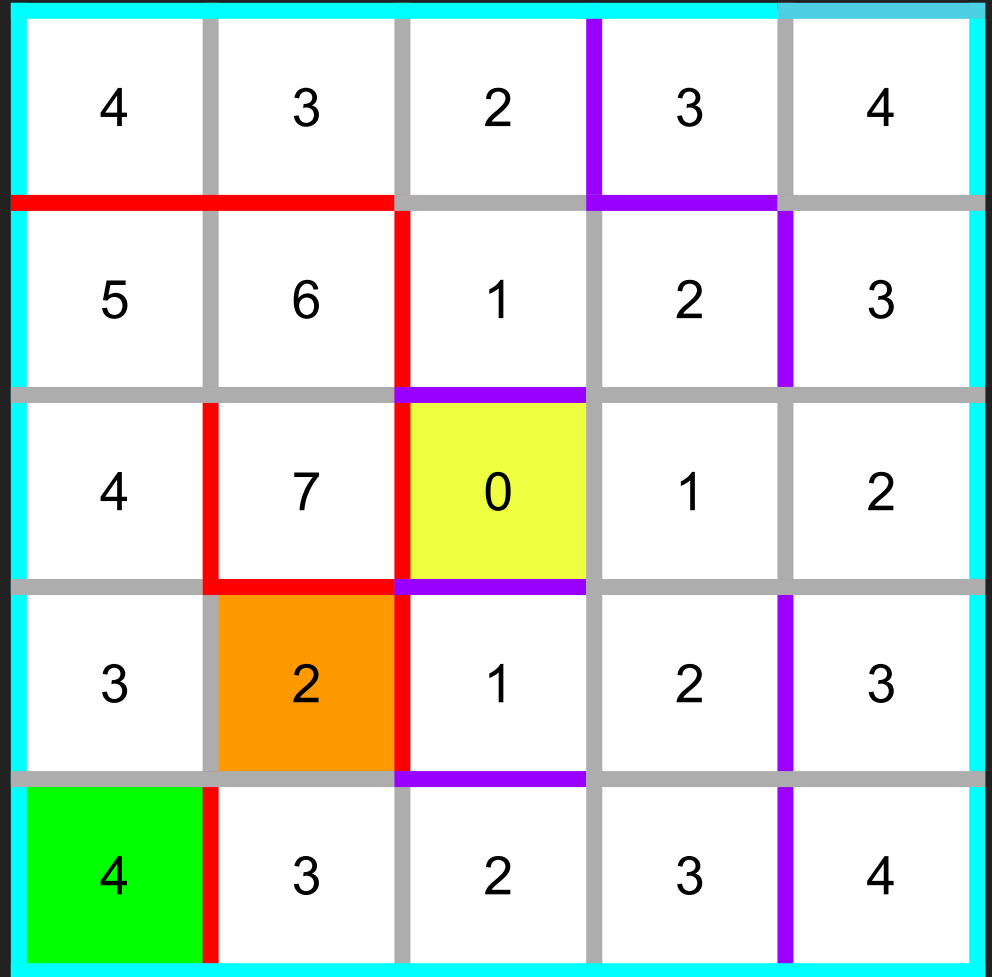
|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 5 | 6 | 1 | 2 | 3 |
| 4 | 7 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 5 | 6 | 1 | 2 | 3 |
| 4 | 7 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 5 | 6 | 1 | 2 | 3 |
| 4 | 7 | 0 | 1 | 2 |
| 3 | 2 | 1 | 2 | 3 |
| 4 | 3 | 2 | 3 | 4 |

# Update

Mouse reverses and travels all the way to this cell

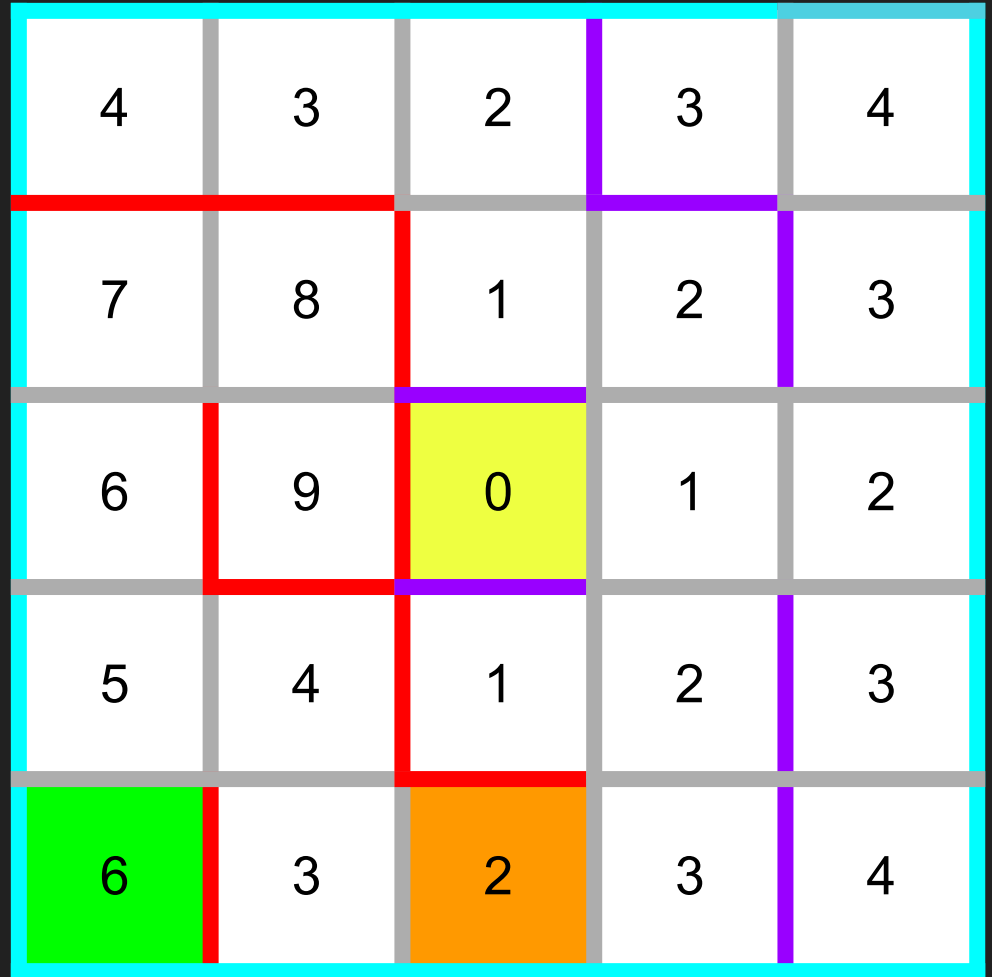


|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 7 | 8 | 1 | 2 | 3 |
| 6 | 9 | 0 | 1 | 2 |
| 5 | 4 | 1 | 2 | 3 |
| 6 | 3 | 2 | 3 | 4 |

|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 3 | 2 | 3 | 4 |
| 7 | 8 | 1 | 2 | 3 |
| 6 | 9 | 0 | 1 | 2 |
| 5 | 4 | 1 | 2 | 3 |
| 6 | 3 | 2 | 3 | 4 |

# Update

Minimum distances > current cell distance so must update cells





|   |    |   |   |   |
|---|----|---|---|---|
| 4 | 3  | 2 | 3 | 4 |
| 9 | 10 | 1 | 2 | 3 |
| 8 | 11 | 0 | 1 | 2 |
| 7 | 6  | 1 | 2 | 3 |
| 8 | 5  | 4 | 3 | 4 |

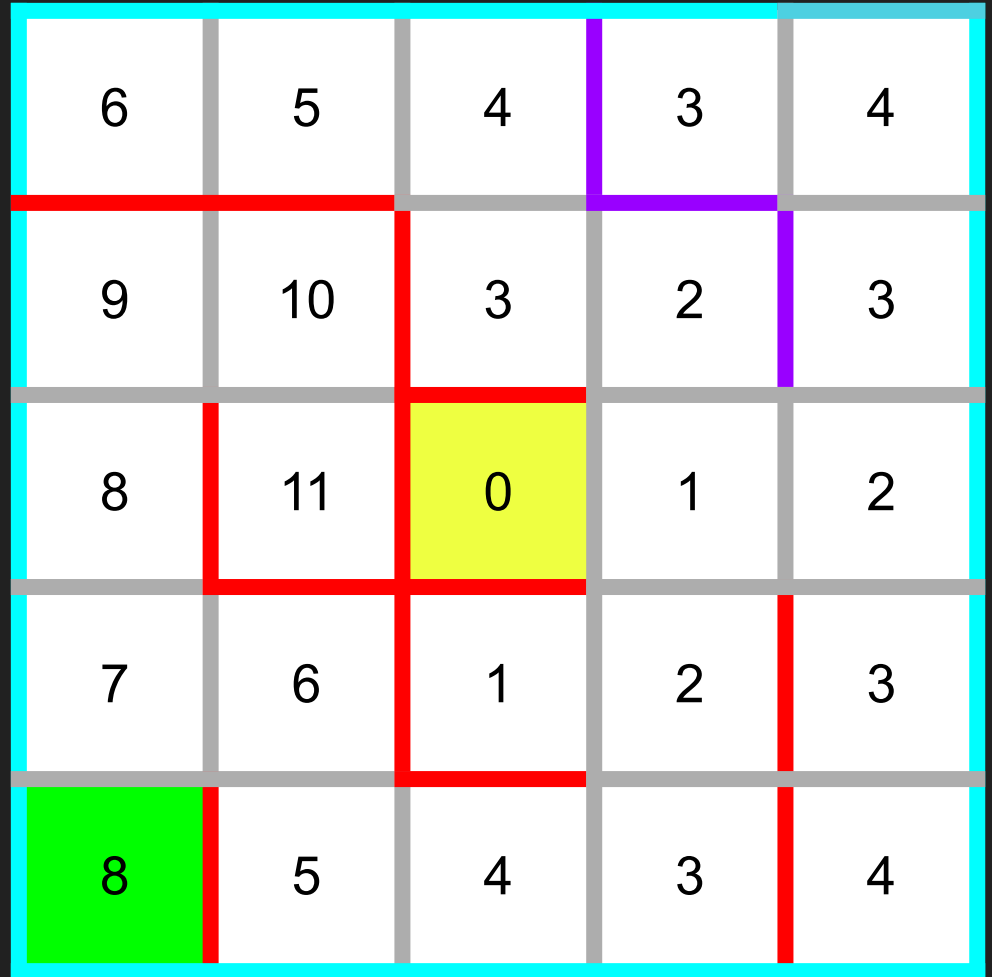
|   |    |   |   |   |
|---|----|---|---|---|
| 4 | 3  | 2 | 3 | 4 |
| 9 | 10 | 1 | 2 | 3 |
| 8 | 11 | 0 | 1 | 2 |
| 7 | 6  | 1 | 2 | 3 |
| 8 | 5  | 4 | 3 | 4 |

|   |    |   |   |   |
|---|----|---|---|---|
| 4 | 3  | 2 | 3 | 4 |
| 9 | 10 | 1 | 2 | 3 |
| 8 | 11 | 0 | 1 | 2 |
| 7 | 6  | 1 | 2 | 3 |
| 8 | 5  | 4 | 3 | 4 |

|   |    |   |   |   |
|---|----|---|---|---|
| 4 | 3  | 2 | 3 | 4 |
| 9 | 10 | 1 | 2 | 3 |
| 8 | 11 | 0 | 1 | 2 |
| 7 | 6  | 1 | 2 | 3 |
| 8 | 5  | 4 | 3 | 4 |

# Finish

Final distance values and walls recorded





## Additional Resources

[IEEE Discord](#)

[IEEE Website](#)

[Micromouse Rules](#)